Azure Update Manager at enterprise scale: a patch framework for 10 to several hundred servers

An expert playbook for building a production patch framework with Azure Update Manager across Azure VMs and Azure Arc-enabled servers: ring design, maintenance configurations as code, reboot policy, dynamic scopes, pre/post events with snapshots and cancellation, compliance reporting, alerting, troubleshooting and rollback for Windows Server and Linux.

9 phases | 29 steps | Tick each item when its "done when" criterion is true

Phase 1 Plan the framework: inventory, ownership, rings and cost

about 2–5 days

- [ ] **1.1 Inventory the estate and draw the support boundary**
  Every server is listed with location (Azure / on-premises / other cloud), OS and version, owner and workload criticality; out-of-scope workloads have a named alternative
- [ ] **1.2 Give each machine exactly one patch orchestrator**
  A decision record states which tool patches which machines during and after migration (Update Manager, Configuration Manager, WSUS-only, or something else) — never two at once
- [ ] **1.3 Design rings and a tag taxonomy that scale from 10 to several hundred servers**
  Ring definitions (for example Pilot, Test, Prod-A, Prod-B), tag names and allowed values are documented and checked against the service limits
- [ ] **1.4 Estimate cost before onboarding Arc machines**
  A cost line exists for Arc-enabled servers, and machines that qualify for no-charge use are identified

Phase 2 Lay the foundation: resource providers, RBAC and network

about 2–4 days

- [ ] **2.1 Register the resource providers in every subscription in scope**
  `Microsoft.Maintenance` is registered everywhere schedules live; Arc subscriptions also have `Microsoft.HybridCompute`, `Microsoft.GuestConfiguration`, `Microsoft.HybridConnectivity` and `Microsoft.Compute`
- [ ] **2.2 Split permissions into operator, schedule owner, automation and reader roles**
  Custom or built-in roles exist for: patch operators (assess/install on demand), schedule owners (maintenance configurations and assignments), automation identities (policy remediation, pre/post events) and read-only reporting; nobody needs Owner for daily patching
- [ ] **2.3 Open the network paths: update sources, Arc endpoints and private connectivity**
  Every machine reaches its update source (Windows Update/Microsoft Update, WSUS, Linux repos) and Arc machines reach the documented Arc endpoints over TLS 1.2+

Phase 3 Onboard Azure VMs and Arc-enabled servers

about 2–10 days (mostly Arc rollout)

- [ ] **3.1 Connect on-premises and other-cloud servers with Azure Arc at scale**
  Machines show as Connected in Azure Arc in the intended resource groups and region, already carrying the ring tags
- [ ] **3.2 Set Azure VMs to Customer Managed Schedules**
  Every Azure VM that belongs to a ring has patch mode `AutomaticByPlatform` and `bypassPlatformSafetyChecksOnUserSchedule` = `true`, visible in the VM's JSON view
- [ ] **3.3 Align the Windows Update client, Group Policy and WSUS with Update Manager**
  No GPO sets conflicting values for Configure Automatic Updates, No auto-restart with logged on users or Always automatically restart at the scheduled time; WSUS approvals are part of the ring calendar

Phase 4 Turn on periodic assessment and take the baseline

about 1–2 days + 1 week of data

- [ ] **4.1 Enforce periodic assessment with Azure Policy**
  Policy assignments with remediation tasks exist for Azure VMs and Arc servers (separate Windows and Linux assignments), and compliance shows no non-compliant in-scope machines
- [ ] **4.2 Record the baseline in Azure Resource Graph**
  A dated export lists, per machine, missing updates by classification, plus the machines that report no assessment at all

Phase 5 Build maintenance configurations and deployment rings as code about 3–5 days

- [ ] **5.1 Anchor the ring calendar to Patch Tuesday with offsets**
  Each ring has a recurrence that lands a fixed number of days after the second Tuesday, in a named time zone, and the calendar is published to application owners
- [ ] **5.2 Create the maintenance configurations from Bicep or Azure CLI**
  Each ring's configuration exists with scope `InGuestPatch`, `extensionProperties.InGuestPatchMode` = `User`, and is deployed from source control
- [ ] **5.3 Choose classifications, inclusions and exclusions per OS**
  Each configuration lists explicit Windows classifications and Linux classifications, and every exclusion has a ticket and an expiry
- [ ] **5.4 Set the reboot policy and size the window to the documented timeline**
  Each ring has `rebootSetting` = `IfRequired`, `NeverReboot` or `AlwaysReboot` by design, windows are long enough for the baseline backlog, and clustered or availability-set members are split across windows

Phase 6 Attach machines at scale: dynamic scopes, Azure Policy and cross-subscription about 1–3 days

- [ ] **6.1 Use tag-driven dynamic scopes for each ring**
  Each ring configuration has a subscription-level dynamic scope filtering on `PatchRing` (and OS type where needed); the preview matches the inventory
- [ ] **6.2 Let Azure Policy keep new machines in a ring**
  One assignment of Schedule recurring updates using Azure Update Manager per ring exists, with a system-assigned identity and remediation, and compliance is reviewed weekly
- [ ] **6.3 Run schedules centrally across subscriptions**
  Ring configurations live in one patching subscription and target machines in the other subscriptions; resource moves trigger a re-assignment step

Phase 7 Pre and post events: snapshots, health gates and safe cancellation about 2–4 days

- [ ] **7.1 Wire pre and post events through Event Grid**
  Each production configuration has an Event Grid system topic with subscriptions for `Microsoft.Maintenance.PreMaintenanceEvent` and `Microsoft.Maintenance.PostMaintenanceEvent`, delivering to a webhook (Automation runbook) or an Azure Function
- [ ] **7.2 Respect the pre-event timeline**
  Pre-events finish within 20 minutes, cancellation (if needed) is called at least 10 minutes before the window, and schedule edits with a pre-event happen 40+ minutes before start
- [ ] **7.3 Implement the standard pre/post playbook**
  Pre: power on, snapshot, stop or drain services, health gate. Post: start services, health check, tag/report, power off if it was off

Phase 8 Operate: compliance reporting, alerting and out-of-band patching about 2–3 days, then monthly

- [ ] **8.1 Build compliance reporting on Resource Graph and workbooks**
  A workbook or dashboard shows per ring: machines assessed in the last 24 hours, missing critical/security updates, last successful run, failed and pending patch counts
- [ ] **8.2 Alert on failures, missing assessments and drift**
  Alert rules notify the owning team on failed runs, machines with pending critical/security updates beyond your SLA, and missing assessments
- [ ] **8.3 Handle zero-days and out-of-band releases**
  A documented emergency path exists: one-time update with a specific KB or package, compressed ring order, and verification with an independent scanner
- [ ] **8.4 Fold in hotpatching and Extended Security Updates**
  Hotpatch-eligible servers are enrolled and on their own faster schedule; ESU servers are enrolled and visible in the same reports

Phase 9 Troubleshooting, rollback and the pitfalls that break schedules about Ongoing

- [ ] **9.1 Triage failed runs from the right logs**
  Runbooks name the log locations and the common causes; first-line staff can classify a failure as prerequisite, connectivity, window or OS issue

- [ ] **9.2 Roll back safely: stop the ring, exclude, restore**
  For a bad update the team can, in order: stop later rings, exclude the KB or package, restore or uninstall on affected machines, and prove the rollback worked
- [ ] **9.3 Review the pitfalls every quarter**
  A quarterly review confirms none of the known pitfalls has crept back in, and the framework's tags, policies and configurations still match the inventory

Owner	Change / ticket	Date completed

Original checklist by Tenant Recipes, an independent project not affiliated with Microsoft or any vendor. Every step, its actions, pitfalls and the official sources it was checked against are on https://tenantrecipes.github.io/guides/azure-update-manager-enterprise-patching.html. Time estimates are our own planning figures. Confirm against the linked sources and test in a pilot before production.